Java Backend Developer Interview Answers

Java 8/17 • Core Java • Streams • Spring Boot • Microservices • Kafka • SQL • JPA/Hibernate • REST • Concurrency • SonarQube • React/JavaScript

For 3 YOE interviews: learn the direct answer first, then practice explaining internals, trade-offs, failure scenarios and how the concept was used in your project.

Core Java & Java 8/17

1. What are important Java 17 features?

Java 17 is an LTS release. Important language features around this generation include records, sealed classes, pattern matching for instanceof, switch expressions, text blocks and enhanced random APIs. It also provides a newer JVM, security fixes, performance improvements and a longer-supported runtime baseline.

2. Why upgrade from Java 8 to Java 17 if we do not use records/sealed classes?

An upgrade is valuable even without new syntax: newer JVM/runtime improvements, security fixes, supported libraries, tooling improvements and a longer LTS support window. It also enables modern Spring Boot and dependency versions.

3. Advantages of Java 17 over Java 8?

Modern language features, newer JVM optimizations, security/runtime improvements, better tooling and LTS support. Java 17 also removes the need to remain on an increasingly old runtime.

4. Major Java 8 features?

Lambda expressions, functional interfaces, Stream API, Optional, method references, default/static interface methods, java.time, CompletableFuture and collection/concurrency improvements.

5. What is a default interface method and why was it introduced?

A default method contains an implementation in an interface. It allowed Java to evolve existing interfaces without forcing every existing implementation to immediately add the new method.

6. What is a static method in Java?

A static method belongs to the class rather than an object. It is useful for stateless utility or factory behavior and can be invoked without creating an instance.

7. What is Stream API?

A Stream is a pipeline for declarative processing of data using operations such as filter, map, sorted, reduce and collect. It does not store data itself and intermediate operations are lazy.

8. What is a method reference?

A compact alternative to a lambda when an existing method already matches the functional interface. Examples: String::trim, Person::getAge, System.out::println.

9. map() vs flatMap()?

map transforms each element into one result. flatMap transforms each element into a stream and flattens all resulting streams into one stream, which is useful for nested collections.

10. What is Optional.ofNullable()?

It creates an Optional containing the value when non-null, otherwise Optional.empty(). Unlike Optional.of(), it safely accepts null.

11. Can an abstract class be final?

No. final prevents inheritance, while abstract requires inheritance to provide a concrete implementation. The two purposes conflict.

12. ArrayList vs LinkedList?

ArrayList uses a dynamic array, giving O(1) average random access and usually better cache locality. LinkedList has node links and O(n) random access; insertion/removal is efficient only when the node/position is already known.

13. Checked vs unchecked exceptions?

Checked exceptions must be handled or declared. Unchecked exceptions extend RuntimeException and do not have that compile-time requirement. In Spring applications, meaningful domain exceptions plus centralized handling are common.

14. String vs StringBuilder vs StringBuffer?

String is immutable. StringBuilder is mutable and normally preferred for repeated string construction in one thread. StringBuffer is synchronized and therefore generally has more overhead.

15. What are generics?

Generics provide compile-time type safety and reusable code, for example List<String>. They reduce explicit casting and catch many type errors before runtime.

16. Serialization vs deserialization?

Serialization converts an object/value into a transferable or storable representation; deserialization reconstructs it. Java serialization exists, but JSON serialization is common for REST APIs.

17. Different ways to create threads?

Thread/Runnable, Callable with ExecutorService, CompletableFuture, and framework-managed executors. Production code normally prefers managed thread pools over creating raw threads per task.

18. What is deadlock and how do you prevent it?

Deadlock occurs when threads wait indefinitely for locks held by each other. Prevent it with consistent lock ordering, smaller critical sections, avoiding unnecessary nested locks, and time-bounded tryLock where appropriate.

19. How does HashMap work internally?

HashMap hashes the key, maps the hash to a bucket, and stores key/value entries there. Collisions are handled within the bucket, with treeification possible for heavily collided bins in modern Java. Average lookup/update is O(1).

20. Why is HashMap not thread-safe?

Concurrent writes can race and produce lost updates or inconsistent state. Use ConcurrentHashMap or external synchronization when shared mutable access is required.

21. How does ConcurrentHashMap work?

Modern ConcurrentHashMap combines CAS and fine-grained synchronization so unrelated bins can often be updated concurrently rather than using one global lock. Reads are highly concurrent.

22. Why doesn't volatile make i++ thread-safe?

volatile provides visibility/order guarantees, but i++ is read + modify + write. Two threads can read the same value and overwrite each other. Use AtomicInteger or synchronization.

23. What happens when two keys have the same hashCode()?

They collide into the same bucket. HashMap then uses equals() to distinguish the keys. Equal objects must have the same hashCode.

24. AtomicInteger vs synchronized?

AtomicInteger is excellent for simple atomic operations such as counters and compare-and-set. synchronized is better when several related variables/actions must be protected as one invariant.

25. When is an object eligible for GC?

When it is no longer reachable from GC roots such as active thread references, static references or JNI references. Eligibility does not mean collection happens immediately.

26. What would you check for OutOfMemoryError?

Identify the OOM type, inspect heap/GC logs, heap dumps, retained objects, allocation rate, cache sizes, large allocations and configured heap limits. Look for memory leaks or unbounded state.

27. wait() vs sleep() vs join()?

wait() releases the object's monitor and waits for notification; sleep() pauses without releasing locks held by the thread; join() waits for another thread to finish.

28. Why are immutable objects useful in multithreading?

Their state cannot change after construction, so shared immutable state needs no synchronization for mutation. This reduces race-condition risk and simplifies caching and concurrent design.

29. How can ThreadLocal cause a memory leak?

Values remain attached to long-lived pool threads if not removed. For request/task-scoped values, use try/finally and ThreadLocal.remove().

30. Why can a race condition still happen with synchronized methods?

Synchronization only protects code using the same monitor. A race can remain if related state is accessed through another lock, outside synchronized regions, or across multiple objects whose invariant is not protected together.

31. How do you detect a production deadlock?

Take thread dumps using tools such as jstack or JVM monitoring, inspect BLOCKED threads and lock ownership, identify the cycle, then fix lock ordering or synchronization design.

32. How does CompletableFuture execute async work?

runAsync/supplyAsync submit work to an executor; by default asynchronous stages commonly use the common ForkJoinPool. thenApply transforms results, thenCompose chains futures, and exceptionally/handle can recover from failures.

33. What happens when a Java class is loaded?

The JVM loads class bytes, links the class through verification/preparation/resolution, then initializes it when required. Static initialization occurs during initialization.

34. How do you troubleshoot long GC pauses?

Inspect GC logs and JVM metrics, pause frequency/duration, allocation rate, old-generation occupancy, heap sizing and retained objects. Use heap dumps and review whether the collector/configuration matches the workload.

35. Why might an application suddenly have thousands of threads?

Possible causes include thread-per-request design, oversized/incorrect executors, blocked I/O, connection-pool problems, retries or thread leaks. Use thread dumps and executor metrics.

Java Streams & Coding

36. Remove duplicates using Streams.

Use list.stream().distinct().collect(Collectors.toList()). For custom objects, distinct depends on correctly implemented equals() and hashCode().

37. Highest-salary employee department-wise?

Group by department and choose the maximum salary: Collectors.groupingBy(Employee::getDepartment, Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))).

38. Duplicate characters and their frequency?

Convert characters to a stream, group with groupingBy(Function.identity(), counting()), then filter entries where count > 1. Use LinkedHashMap if encounter order matters.

39. Group employees by department.

employees.stream().collect(Collectors.groupingBy(Employee::getDepartment)).

40. Second-highest salary using Streams?

employees.stream().map(Employee::getSalary).distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst(). The distinct() requirement should be clarified in an interview.

41. Top 3 salaries?

map salary, optionally distinct, sort descending, then limit(3).

42. Nth highest salary?

map salary → distinct → sort descending → skip(n-1) → findFirst().

43. Even/odd partition?

Collectors.partitioningBy(n -> n % 2 == 0) returns a Map<Boolean,List<Integer>>.

44. List<Employee> to Map<Id,Employee>?

Collectors.toMap(Employee::getId, Function.identity(), (oldValue,newValue) -> newValue) handles duplicate keys explicitly.

45. Stream vs Collection?

A Collection stores data; a Stream describes a computation over data. A collection can generally be traversed repeatedly, while a stream is normally single-use.

46. Intermediate vs terminal operations?

Intermediate operations such as filter/map/sorted return another stream and are lazy. Terminal operations such as collect/count/reduce trigger evaluation.

47. filter() vs map()?

filter selects elements based on a predicate; map transforms each element.

48. map() vs peek()?

map is for transformation. peek is mainly for observing/debugging and should not contain essential business logic.

49. findFirst() vs findAny()?

findFirst respects encounter order when one exists. findAny can return any matching element and can be useful with parallel streams.

50. reduce() vs collect()?

reduce combines elements into a value; collect is designed for mutable reduction into structures such as List, Set and Map.

51. groupingBy() vs partitioningBy()?

groupingBy supports arbitrary groups. partitioningBy creates exactly two groups based on a boolean predicate.

52. Can a Stream be reused?

No. After a terminal operation the stream is consumed. Create a new stream from the source.

53. Sequential vs parallel stream?

Sequential uses one execution path. Parallel splits work across the ForkJoinPool. It is useful only when workload size, computation cost and splitting characteristics justify the overhead.

54. When should you avoid parallelStream()?

Avoid it for small data, blocking I/O, order-sensitive operations, shared mutable state, poor splitting workloads, or when common-pool contention is a concern.

55. How does distinct() work?

It tracks previously seen elements using equality/hash semantics. Therefore equals/hashCode must be consistent. It can require state and memory.

56. How does sorted() work internally?

It is stateful: elements generally must be buffered and sorted before being emitted. This means extra memory and O(n log n) sorting work in the usual case.

57. What happens if a stream lambda throws an exception?

The pipeline terminates and the exception propagates unless handled. Checked exceptions require explicit handling because standard functional interfaces do not declare them.

Spring Boot

58. Why use Spring Boot?

It reduces configuration and boilerplate through starters, auto-configuration, embedded servers, externalized configuration and production-ready features.

59. What is auto-configuration?

Spring Boot examines the classpath, properties and existing beans and automatically configures common infrastructure. It backs off when the application provides its own matching bean.

60. @SpringBootApplication?

It combines @Configuration, @EnableAutoConfiguration and @ComponentScan and is normally placed on the main application class.

61. @ComponentScan?

It scans configured packages for Spring components such as @Component, @Service, @Repository and @Controller and registers them as beans.

62. @Component vs @Service vs @Repository?

All are component stereotypes. @Component is generic, @Service communicates business/service logic, and @Repository communicates persistence logic and supports exception translation.

63. @Bean vs @Component?

@Component is discovered through scanning. @Bean is a method that explicitly creates/configures an object, usually inside a @Configuration class.

64. What is Dependency Injection?

A class receives dependencies from the framework instead of constructing them itself. Constructor injection makes dependencies explicit, supports final fields and is easy to unit test.

65. How does @Transactional work?

Spring usually uses a proxy around the bean. A transactional call starts or joins a transaction, then commits or rolls back based on outcome and rollback rules. Self-invocation can bypass the proxy.

66. How do you handle exceptions in Spring Boot?

Use @RestControllerAdvice with @ExceptionHandler methods to map domain, validation and technical exceptions to consistent HTTP responses.

67. How do you validate REST input?

Use Bean Validation annotations such as @NotBlank, @Email, @Size and @Positive on DTOs with @Valid/@Validated, then handle validation errors centrally.

68. application.properties vs application.yml?

Both externalize configuration. Properties uses key=value; YAML uses indentation and nested structures. YAML is often convenient for hierarchical configuration.

69. How do you handle transactions?

Put @Transactional at an appropriate service boundary, keep transactions short, select propagation/isolation deliberately and avoid unnecessary external network calls inside DB transactions.

70. How does Spring Security work?

HTTP requests pass through a security filter chain. Authentication establishes the principal, then authorization checks access based on roles/authorities. The SecurityContext holds authentication for the request.

71. How is JWT authentication implemented?

Authenticate credentials, issue a signed JWT with claims and expiration, send it over HTTPS, validate signature/expiry on each request, and populate the SecurityContext with the authenticated principal.

Microservices

72. Explain microservices architecture.

Services are independently deployable units organized around business capabilities. They may own their data and communicate using REST/gRPC or events. Benefits include independent scaling/deployment; trade-offs include network failures, distributed consistency and operational complexity.

73. What is a circuit breaker?

It prevents repeated calls to an unhealthy dependency. Typical states are closed, open and half-open. It fails fast while a dependency is unhealthy and later tests recovery.

74. What is service discovery?

A registry maintains service instances and allows clients to find available instances dynamically instead of hard-coding host/port information.

75. How does a service register with discovery?

The service starts with a discovery client, provides registry configuration and service identity, registers its instance and usually sends heartbeats/renewals. Exact configuration depends on the discovery system.

76. Service A calls Service B: complete discovery flow?

A registers with the registry. A's client looks up B, receives an available instance, sends the request, then applies timeout/retry/circuit-breaker behavior if configured.

77. Synchronous vs asynchronous communication?

Synchronous REST/gRPC waits for a response. Asynchronous Kafka-style messaging decouples producer and consumer and is useful for events/workflows, but introduces eventual consistency and more complex failure handling.

78. How do you handle service failure?

Use timeouts, bounded retries with backoff, circuit breakers, bulkheads, health checks, idempotency, fallbacks where meaningful, and observability.

79. Retry vs circuit breaker?

Retry repeats a failed call for transient failures. Circuit breaker stops calls after repeated failures to prevent cascading overload. They should be configured together carefully.

80. What is Saga?

Saga coordinates a distributed business transaction as a sequence of local transactions, with compensating actions when later steps fail. It avoids a distributed DB transaction but introduces eventual consistency.

81. How do you maintain consistency between services?

Use service-owned data, events, Saga workflows, idempotent consumers, transactional outbox where appropriate, retries/DLQ and reconciliation. Avoid sharing a database just to mimic a monolith.

82. What is API Gateway?

A gateway is a common entry point that routes requests and can centralize authentication, rate limiting, logging, routing and other cross-cutting concerns.

83. How do you handle failures between microservices?

Use timeouts, bounded retries, circuit breakers, correlation IDs, fallback where appropriate, health checks and metrics. For messaging, use retries and DLQs.

Kafka

84. What is Spring Kafka?

Spring Kafka integrates Apache Kafka with Spring abstractions, simplifying producers, consumers, listener containers, serialization, error handling, retries and configuration.

85. What is a Kafka consumer?

A consumer reads records from topic partitions, tracks offsets and processes records. Consumers are commonly organized into consumer groups.

86. What is a consumer group?

Consumers in the same group cooperate to process partitions. A partition is assigned to at most one consumer in that group at a time. Different groups can consume the same topic independently.

87. How are partitions assigned to consumers?

Kafka assigns partitions among consumers in a group using a partition assignment strategy. Rebalances occur when group membership or assignments change.

88. Can different consumer groups consume the same partition?

Yes. Each group has independent offsets, so the same partition can be consumed independently by multiple groups.

89. What if a consumer crashes before committing its offset?

Another consumer can resume from the last committed offset after rebalance, so the failed record may be processed again. This is why idempotent processing is important.

90. What is a Kafka partition?

An ordered append-only log within a topic. Partitions provide scalability and parallelism. Ordering is guaranteed within a partition, not across all partitions.

91. What is a Dead Letter Topic?

A topic used to store records that cannot be processed after configured retries or because the error is non-retryable. It lets normal processing continue while failures are investigated.

92. How do you make Kafka consumers idempotent?

Use a unique event ID/business key, store processed IDs or use DB uniqueness constraints, design writes as upserts where appropriate, and commit offsets at the correct point.

93. What if the database crashes while consuming Kafka?

If processing fails before offset commit, the message can be delivered again after recovery. Use retries/backoff and idempotent DB operations; patterns such as transactional outbox help with reliable event publication.

94. At-most-once vs at-least-once vs exactly-once?

At-most-once may lose messages but avoids redelivery. At-least-once may redeliver, so consumers must tolerate duplicates. Exactly-once is possible for supported Kafka workflows, but external DB side effects need additional design.

SQL & JPA/Hibernate

95. Types of SQL joins?

INNER returns matching rows; LEFT keeps all left rows plus matches; RIGHT keeps all right rows plus matches; FULL OUTER returns all rows from both sides where supported; CROSS produces a Cartesian product.

96. WHERE vs HAVING?

WHERE filters rows before grouping. HAVING filters groups after GROUP BY and is normally used with aggregate conditions.

97. Second-highest salary SQL?

For the second distinct salary: SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee). DENSE_RANK() is more flexible for general ranking.

98. How do indexes improve performance?

Indexes provide a structure for locating rows efficiently for common filters, joins and ordering. They consume storage and slow writes, so index actual query patterns rather than every column.

99. How do you troubleshoot a slow query?

Inspect execution plans, row counts/cardinality, indexes, joins, scans, sorts, locks and statistics. Look for missing indexes, non-sargable predicates, excessive rows and N+1 query patterns.

100. What is lazy vs eager loading?

Lazy fetches an association when accessed; eager fetches it earlier. Lazy can reduce over-fetching but may cause N+1 or lazy-initialization issues; eager can load too much data.

101. What is N+1 query problem?

One query loads N parent rows and then an additional query executes for each parent to fetch related data, creating many DB round trips.

102. How do you resolve N+1?

Use JOIN FETCH carefully, EntityGraph, batch fetching, projections or dedicated queries. Verify generated SQL because joining multiple collections can create large result sets.

103. save() vs saveAndFlush()?

save persists/merges through the persistence context; SQL may execute later at flush/commit. saveAndFlush explicitly flushes immediately but does not by itself commit the transaction.

104. What is JOIN FETCH?

JPQL JOIN FETCH retrieves an association in the same query and initializes it, commonly to avoid N+1.

105. What is cascade?

Cascade controls whether persistence operations such as persist, merge or remove propagate from parent to related entities. Use cascade types deliberately.

106. What is orphanRemoval?

When enabled for supported one-to-one/one-to-many relationships, removing a child from the parent's relationship can cause that child row to be deleted. It should be used carefully.

107. Optimistic vs pessimistic locking?

Optimistic locking uses a version value such as @Version and detects conflicting updates. Pessimistic locking obtains database locks and can block other transactions. Choose based on contention and consistency needs.

108. PostgreSQL MVCC vs MySQL?

Both use MVCC, but their implementation details differ. PostgreSQL stores row versions in tuples and relies on VACUUM; InnoDB uses undo information and purge. Avoid saying one database does not use MVCC.

REST API & Communication

109. What is WebSocket?

A persistent full-duplex connection where both client and server can send messages after connection establishment. It is useful for chat, live dashboards and real-time notifications.

110. What is WebClient?

Spring WebClient is a non-blocking HTTP client from Spring WebFlux and supports reactive/asynchronous I/O.

111. RestTemplate vs WebClient?

RestTemplate is synchronous/blocking. WebClient is non-blocking/reactive and is preferred for new reactive applications; RestTemplate remains common in traditional MVC code.

112. PUT vs PATCH?

PUT generally replaces a resource representation and is designed to be idempotent. PATCH applies partial modifications; idempotency depends on the patch semantics.

113. Common HTTP status codes?

200 OK, 201 Created, 202 Accepted, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

114. What is idempotency?

Repeating an operation has the same intended final effect as performing it once. PUT and DELETE are designed to be idempotent; POST is not inherently. Payment/order APIs often use idempotency keys.

115. How do you secure REST APIs?

Use HTTPS, authentication/authorization, input validation, least privilege, secure secrets, rate limiting where appropriate, safe errors, audit logging and protection against common web vulnerabilities.

116. How do you version APIs?

Use URI, headers/media types or other documented mechanisms. Maintain backward compatibility where possible and provide a clear deprecation path.

Code Quality, Testing & GitHub

117. What is SonarQube used for?

Static analysis for bugs, vulnerabilities, code smells, duplication and maintainability metrics. CI quality gates can prevent poor-quality changes from progressing.

118. How do you implement code coverage?

Write unit/integration tests, generate coverage with a tool such as JaCoCo, publish the report to CI/SonarQube and configure meaningful quality thresholds.

119. What does code coverage tell you?

Which code paths were executed by tests. It is useful for finding untested areas but does not prove that tests are correct or meaningful.

120. How do you configure GitHub branch protection?

Protect the target branch by requiring pull requests, approvals and successful CI checks, and optionally signed commits/linear history or restrictions on force pushes. Exact options depend on GitHub configuration.

121. What branching strategies are common?

GitHub Flow, trunk-based development and Git Flow are common approaches. Choose based on release cadence, team size and deployment model.

React.js & JavaScript

122. What are React Hooks?

Functions that let functional components use state and other React capabilities. Common hooks include useState, useEffect, useContext, useRef, useMemo and useCallback.

123. What is React Router?

A client-side routing library that maps URLs to React components without requiring full page reloads.

124. What is Redux?

A state-management library that centralizes shared application state and updates it through actions/reducers. It is useful when many parts of the UI need coordinated state.

125. useRef vs useCallback vs useMemo?

useRef stores a mutable value across renders without causing a render when changed. useCallback memoizes a function reference. useMemo memoizes a computed value. Use them when they solve a real performance/reference-stability problem.

126. BrowserRouter and useNavigate?

BrowserRouter manages client-side routes using the History API. useNavigate provides programmatic navigation, for example after successful login.

127. What is a JavaScript closure?

A function retains access to variables from its lexical scope even after the outer function has returned.

128. What is hoisting?

Declarations are processed before execution. var is initialized to undefined; let/const are in the temporal dead zone until initialization; function declarations have their own hoisting behavior.

Real-World Scenario Questions

129. How would you investigate an API that changed from 100 ms to 4–5 seconds?

Use traces/metrics to split latency into application, DB, network and downstream time. Check thread pools, connection pools, GC pauses, DB locks/execution plans, downstream services and recent deployments. Compare a healthy trace with a slow trace.

130. Why can requests time out while the application is not crashing?

The process can be alive while all useful threads are blocked or pools are exhausted. Common causes are DB connection exhaustion, lock waits, slow downstream services, blocked I/O or executor queue saturation.

131. Why can a shared HashMap fail in production but pass tests?

Concurrency timing is nondeterministic. Unsynchronized updates can race and corrupt application-level state even when ordinary tests do not reproduce the interleaving. Use ConcurrentHashMap or correct synchronization.

132. How do you investigate concurrency issues?

Capture thread dumps, identify shared mutable state, inspect locks and executors, reproduce under load, add concurrency tests, and monitor queue/thread/lock contention.

133. How do you choose thread-pool size?

CPU-bound work often starts near available processor count. I/O-bound work can use more threads, but DB connections and downstream capacity usually become limits. Load-test and tune using CPU, queue, latency and dependency metrics.

134. How do you handle database failure during Kafka consumption?

Do not commit the Kafka offset if DB processing failed. Retry transient failures with backoff, send unrecoverable messages to a DLT, and make DB processing idempotent so redelivery is safe.

135. How do you explain your project architecture?

Describe one complete request path: client → gateway → controller → service → repository → database. Then explain inter-service calls, Kafka events, security, discovery, resilience, logging/monitoring and deployment. Finish with one real feature end-to-end.

136. How should a 3-YOE candidate answer scenario questions?

Use Situation → Problem → Investigation → Decision → Implementation → Result → Trade-off. Be clear about what you personally implemented and do not claim tools or production experience you did not have.

Final-Day Revision Checklist

• Core Java: OOP, equals/hashCode, collections, exceptions, generics, String immutability, threads, locks, volatile, atomics, HashMap and ConcurrentHashMap.

• Java 8: Lambda, functional interfaces, Streams, Optional, method references, CompletableFuture and default/static interface methods.

• Java 17: LTS benefits, records, sealed classes, pattern matching, switch expressions and JVM/runtime improvements.

• Spring Boot: IoC/DI, beans, component scanning, auto-configuration, validation, exception handling, transactions and Spring Security/JWT.

• Microservices: discovery, gateway, REST, Kafka, circuit breaker, retry, timeout, Saga, idempotency and observability.

• Kafka: partitions, consumer groups, offsets, rebalancing, DLT, retries, delivery semantics and DB-failure handling.

• SQL/JPA: joins, GROUP BY/HAVING, indexes, execution plans, second-highest salary, lazy/eager loading, N+1, JOIN FETCH and locking.

• REST: HTTP methods/status codes, idempotency, API security, versioning and WebClient.

• Production scenarios: slow API, DB outage, deadlock, GC pause, thread explosion, connection-pool exhaustion and Kafka redelivery.

• Project explanation: one feature end-to-end plus one inter-service/Kafka flow.